

An abstract graphic on a dark blue background. It features a dense cluster of colorful lines (blue, green, yellow, purple, orange) and spheres (yellow, orange, blue, purple, green) in the upper left quadrant. The lines extend across the entire page, creating a complex, web-like pattern.

HUMUSOFT®

MATLAB TOOLBOX

PHX

User Guide – object-oriented rigid-body physics
simulation inside MATLAB

phx toolbox · MATLAB R2026a · guide version 1.0 · June 2026

• Contents

- 1 What PHX is
- 2 Installation and first run
- 3 How PHX thinks – core concepts
- 4 Your first scene: a falling ball
- 5 Body shapes
- 6 Body properties and motion
- 7 Joints and constraints
- 8 Springs and dampers
- 9 Forces, actuators and fields
- 10 Measuring and logging data
- 11 Automation and control: Script and Function
- 12 What PHX can do – richer scenes
- 13 Interactive viewer
- 14 Ready-made examples (demos)
- 15 A complete example: self-balancing robot
- 16 Quick reference

1 What PHX is

PHX is a MATLAB library that lets you assemble a physical scene from rigid bodies, joints and forces, run it in time, and watch the result directly in a MATLAB 3-D plot.

Under the hood the motion is computed by the well-known **Bullet** physics engine, but as a user you never touch it directly. You work only with the clean `phx.*` objects – you create a body, give it a shape, optionally link it to other bodies, and run the simulation. The library takes care of collisions, gravity, friction and rendering.

What PHX is good for:

- **Mechanics and robotics** – pendulums, manipulators, chassis, gear trains, conveyors.
- **Teaching and demonstrations** – gravity, orbits, collisions, stack stability.
- **Control design and testing** – a scene can be driven from a script or from Simulink; sensors and actuators are built in.
- **Rapid prototypes** – a typical scene is 5–10 lines of code.

Who this guide is for. It is written from the user's point of view – it shows *how* PHX is used, through simple examples. It does not describe the internal implementation or the engine programming interface. Every picture in this guide was produced by running the accompanying code.

2 Installation and first run

PHX is provided as a MATLAB toolbox (it requires **MATLAB R2026a**). Once it is installed, the `phx` package is on the path and ready to use – just run one of the ready-made examples:

```
% run one of the ready-made examples
phxex_minimal1
```

All examples live in the `examples/` folder, from minimal scenes to advanced ones (control, optimization, magnetism); they are all named `phxex_*`.

Some examples expect to be run from their own folder (because of textures and STL models). In that case switch the working directory first, e.g. `cd examples`.

3 How PHX thinks – core concepts

Every scene is built from just a handful of object types. Once you understand them, the rest is only a matter of combining them.

Object	What it represents
<code>phx.Simulation</code>	The whole scene and its "engine". Holds time and gravity, and steps the computation.
<code>phx.Body</code>	A rigid body – it has a position, mass, velocity and a shape. The basic building block.
<code>phx.shape.*</code>	The geometry a body "wears" – box, sphere, cylinder, STL model ...
Joints <code>phx.*Joint</code>	A connection of two bodies with restricted motion (hinge, ball joint, gear).
Forces and fields	<code>Spring</code> , <code>Thruster</code> , <code>Resistance</code> , <code>Monopole</code> , <code>Dipole</code> – springs, actuators, environmental drag, gravitational/magnetic interaction.
Observers	<code>Logger</code> (data logging), <code>Trace</code> (motion trail), <code>Measure</code> (distances and velocities).
Automation	<code>Script</code> (prescribed value profiles over time), <code>Function</code> (custom computation/control at every step).

Three steps of every scene

1. **Build** – create the bodies and their links (in any MATLAB axes).
2. **Run** – create a `phx.Simulation` and call `step`.
3. **Evaluate** – read body properties or data from a `Logger`.

Stepping – the `step` method. The most important simulation command:

```
sim.step(interval, substeps, redrawStep)
```

- **interval** – how many seconds to simulate,
- **substeps** – into how many sub-steps the interval is split (computation granularity),
- **redrawStep** – how often the view is redrawn (1 = every step).

A smaller step (more *substeps*) = a more stable but slower computation. Tightly coupled scenes call for a finer step.

4 Your first scene: a falling ball

The simplest meaningful scene: a ball drops onto a solid floor. It shows everything essential – a body, a static body, a shape and how to run things.

```
% 3-D view and lighting
clf; view(3); axis equal; grid on; camlight headlight;

% falling ball (dynamic and massive by default)
phx.Body("Position", [0 0 3], "Shape", {"Sphere", "Color", [.2 .5 .9]});

% static floor (never moves, only there for collisions)
phx.Body("Type", "static", "Position", [0 0 0], ...
        "Shape", {"Box", "Size", [6 6 0.3]});

% assemble and run: 1.5 s in 150 steps
sim = phx.Simulation;
sim.step(1.5, 150, 1);
```

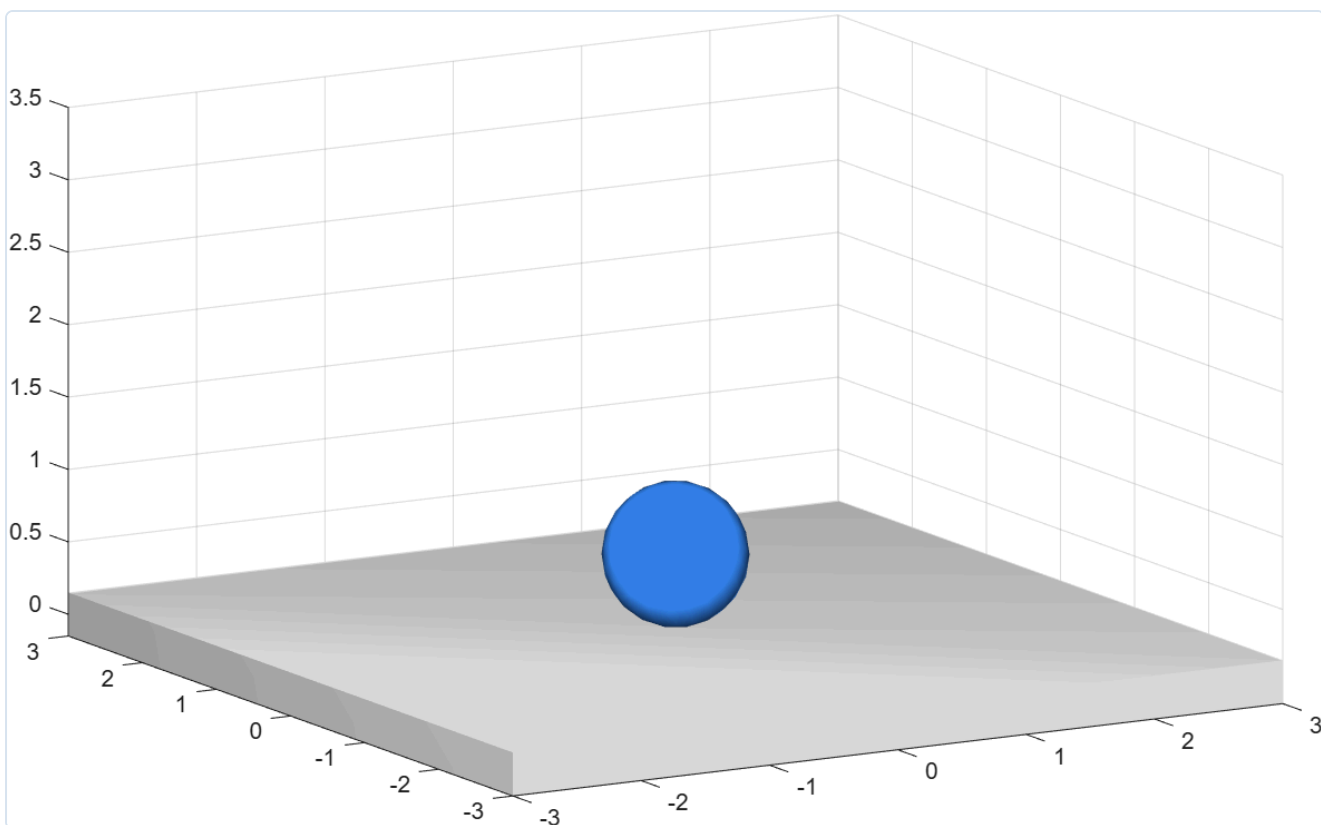


Fig. 1 – After landing, the ball comes to rest on the static floor. Gravity and collisions are handled automatically.

Tip. If you do not set a body's `Type`, it is *dynamic* – it falls, collides and responds to forces. A *static* body never moves (floors, walls, frames). A *kinematic* body moves only in the way you prescribe (conveyors, arms).

5 Body shapes

A shape (`Shape`) determines how a body looks *and* how it behaves in collisions. It is given either as a `phx.shape.*` object, or in shorthand as a cell array `{"Box", "Size", ...}`.

Shape	Main options	Note
<code>Box</code>	<code>Size [x y z]</code>	Rectangular box.
<code>Sphere</code> / <code>Globe</code>	<code>Diameter</code> , <code>Division</code>	Sphere (Globe has better texture mapping).
<code>Cylinder</code>	<code>Diameter</code> , <code>Height</code> , <code>Axis</code>	Cylinder.
<code>Cone</code>	<code>Diameter</code> , <code>Height</code> , <code>Axis</code>	Cone.
<code>Capsule</code>	<code>Diameter</code> , <code>Height</code> , <code>Axis</code>	Cylinder with rounded caps.
<code>STL</code> / <code>OBJ</code>	<code>Source</code> , <code>Scale</code> , <code>Envelope</code>	Import a model from a file.
<code>Terrain</code> , <code>Rock</code> , <code>Extrusion</code> , <code>Revolution</code>	height field / profile	Terrain, rock and profile-based shapes.

Appearance is tuned with `Color` , `Style` (`smooth` , `edged` , `wireframe` , `flat`), `Material` and `Texture` :

```
phx.Body("Position", [0.5 0 2], "Shape", {"Sphere", "Style", "wireframe"});
phx.Body("Position", [0 0 0.5], "Shape", {"Cylinder", "Axis", "y", "Style", "edged"});
phx.Body("Position", [2.5 0 0.5], "Shape", {"Cone", "Axis", "z"});
phx.Body("Position", [2.5 0 2], "Shape", {"Capsule", "Axis", "z"});
phx.Body("Position", [1 -1.8 1], "Shape", {"STL", "Source", "res/cat.stl", ...
    "Scale", 0.025, "Material", "shiny"});
```

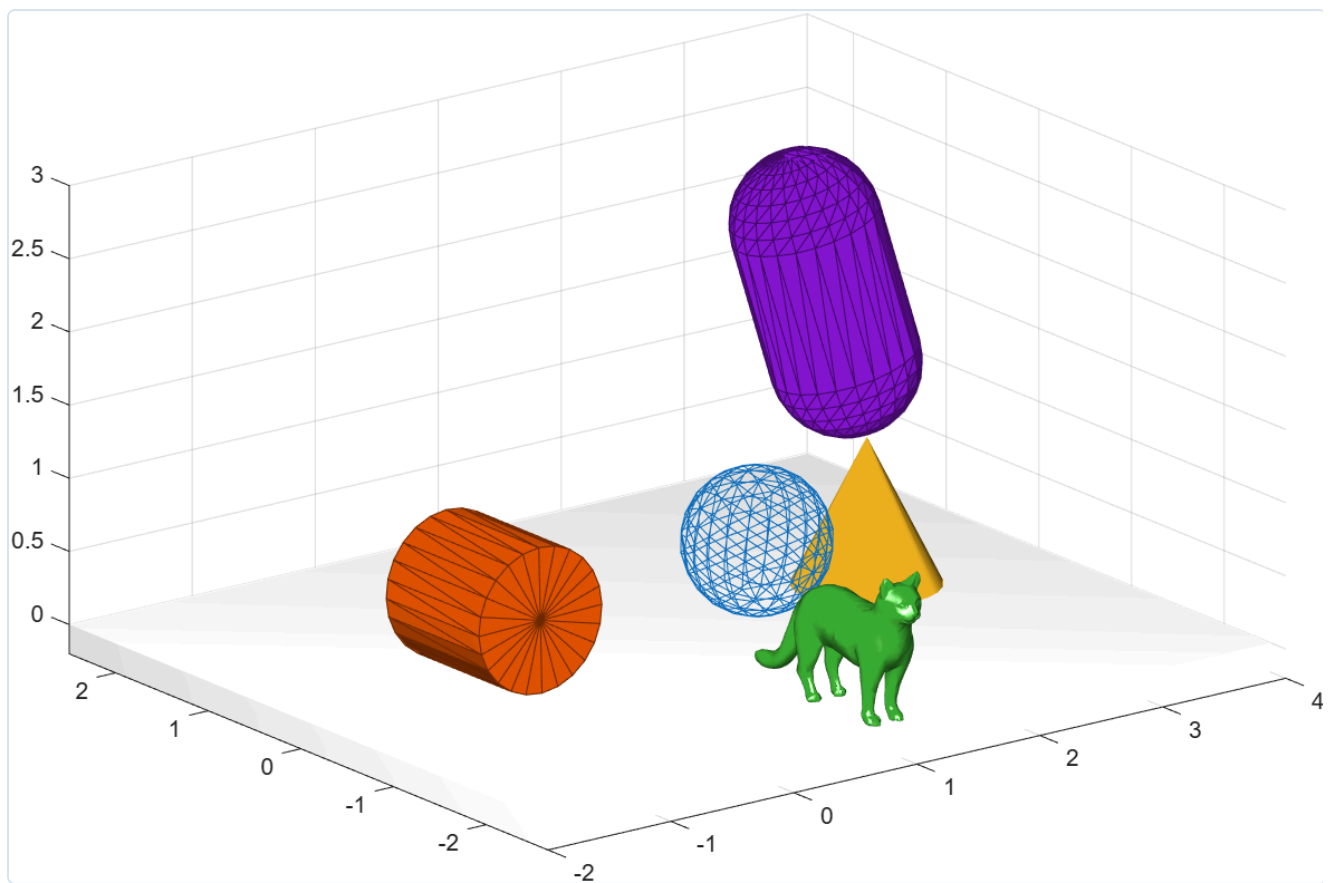


Fig. 2 – Primitive shapes and an imported STL model. The same notation drives both appearance and collision geometry.

6 Body properties and motion

`phx.Body` is the most important object. Its properties can be both read and written – even during the simulation (they read/write the engine directly).

Property	Meaning
<code>Position</code>	Position [x y z] in global coordinates.
<code>Orientation</code> / <code>EulerAngles</code> / <code>Quaternion</code>	Body orientation (rotation matrix, Euler angles or quaternion).
<code>LinearVelocity</code> / <code>AngularVelocity</code>	Linear and angular velocity.
<code>Type</code>	<code>"dynamic"</code> · <code>"static"</code> · <code>"kinematic"</code> .
<code>Mass</code>	Mass [kg]. Can also be derived from the shape density (<code>Density</code>).
<code>Friction</code>	Friction coefficients [slide · roll · spin].
<code>TotalForce</code> / <code>TotalTorque</code> / <code>Energy</code>	Current force, torque and kinetic energy (read-only).

A body can also be acted on directly with a force or a torque:

```
body.applyForce([0 0 50]);    % a 50 N force along +z
body.applyTorque([0 0 60]);   % a torque about the z axis
body.LinearVelocity = [2 0 0]; % set velocity at runtime
```

Coordinate system. PHX uses the **z** axis pointing up by default; the default gravity is `[0 0 -9.81]` m/s². You change it when creating the simulation: `phx.Simulation("Gravity", [0 0 0])` for weightlessness.

7 Joints and constraints

Joints connect two bodies and restrict their relative motion. They are created by writing `phx.TypeJoint(bodyA, bodyB, ...)`; the attachment points are given in the local coordinates of each body (`PointA`, `PointB`).

Joint	Degrees of freedom	Use
<code>FixedJoint</code>	0 – rigid connection	Gluing two bodies together.
<code>RevoluteJoint</code>	1 – rotation about an axis	Hinge, pin, pendulum (<code>AxisA</code> , <code>AxisB</code>).
<code>SphericalJoint</code>	3 – rotation about all axes	Ball joint.
<code>GearJoint</code>	velocity coupling	Gear train with a <code>Ratio</code> .
<code>GenericJoint</code>	6 – with limits	General constraint with translation/rotation limits.

```
A = phx.Body("Type", "static", "Position", [1 1 0], "Shape", {"Box"});
B = phx.Body("Position", [4 1 0], "Shape", {"Box"});
C = phx.Body("Position", [4 -2 0], "Shape", {"Box"});

% a hinge between A and B
phx.RevoluteJoint(A, B, "PointA", [1.5 0 0], "PointB", [-1.5 0 0]);
% a ball joint between B and C
phx.SphericalJoint(B, C, "PointA", [0 -1.5 0], "PointB", [0 1.5 0]);
```

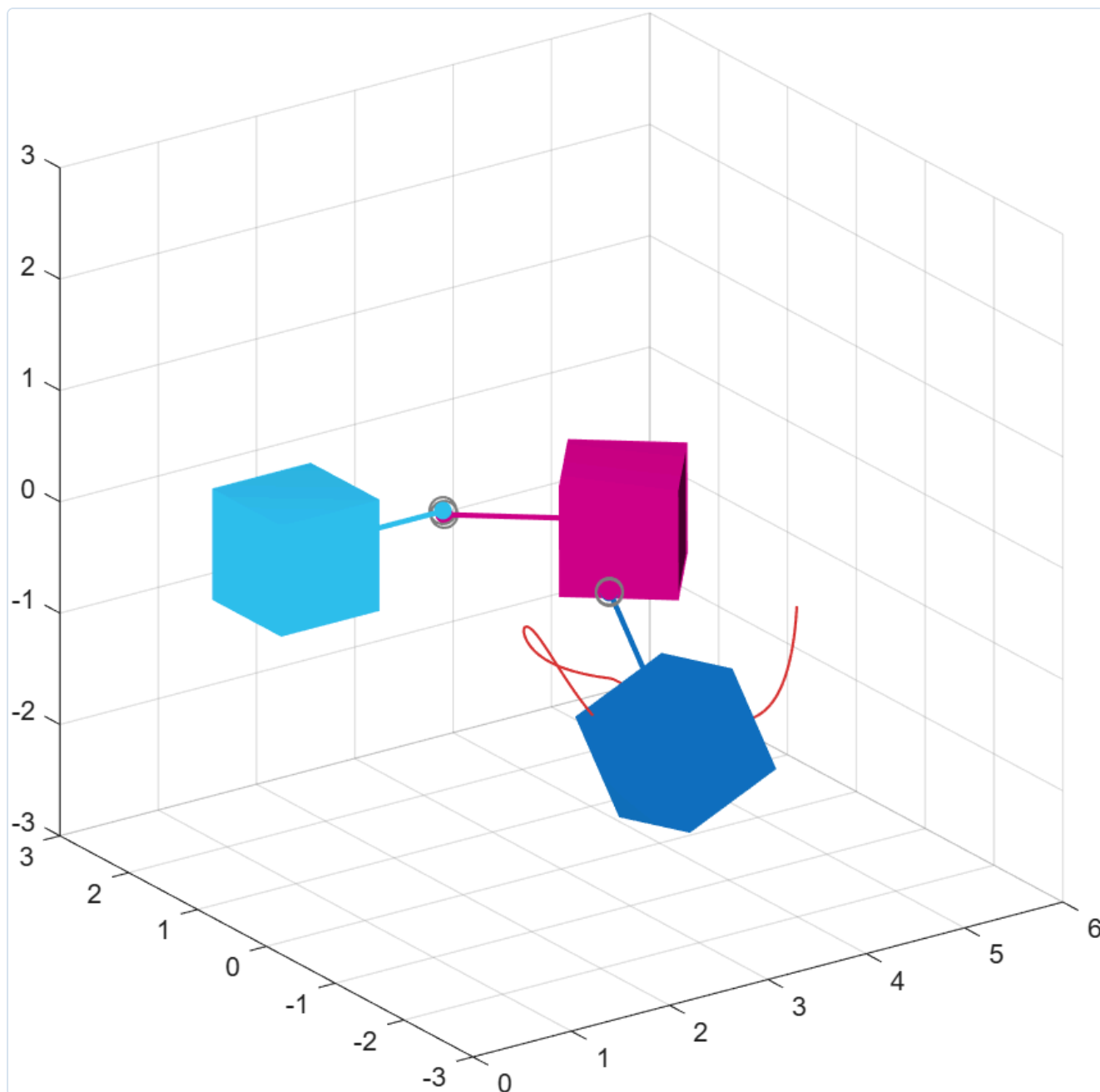


Fig. 3 – A chain of two joints (hinge + ball joint). The red line is the trail of the chain's end (`phx.Trace`).

8 Springs and dampers

`phx.Spring` links two bodies with an ideal spring and a damper. You set the stiffness (`Stiffness`), damping (`Damping`) and attachment points. The force can even be visualized in color.

```
boxes(1) = phx.Body("Type", "static", "Position", [-18 0 10], "Shape", {"Box", "Size", [4  
2 1]});  
for i = 2:7  
    boxes(i) = phx.Body("Position", [(i-1)*6-18 0 10], "Shape", {"Box", "Size", [4 2 1]});  
    phx.Spring(boxes(i-1), boxes(i), "Stiffness", 3e5, "Damping", 2e3, ...  
        "PointA", [2 0 0], "PointB", [-2 0 0], "Colormap", "jet");  
end  
boxes(end).Type = "static";           % anchor the far end too
```

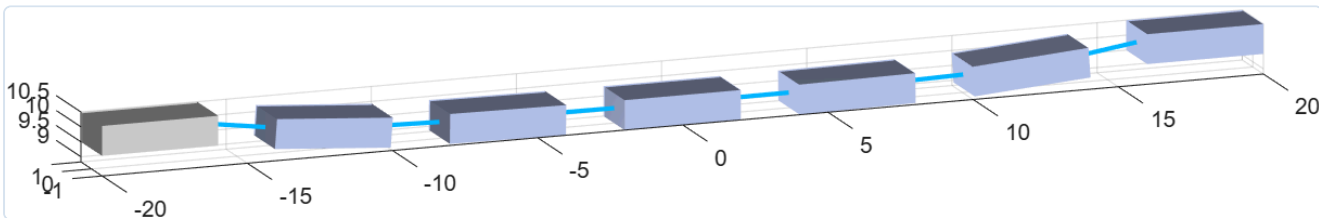


Fig. 4 – A chain of boxes joined by springs, anchored at both ends. The springs sag under their own weight.

For more complex couplings there is `phx.GenericSpring` (a spring/damper on all 6 axes with limits).

9 Forces, actuators and fields

Beyond gravity and contacts, PHX offers ready-made elements for common types of interaction.

Element	What it does	Key options
Thruster	An actuator rigidly attached to a body (rocket, propeller).	Direction , MaxThrust , Throttle (-1...1)
Resistance	Velocity-dependent environmental drag (air, water).	VelocityFactors
Monopole	Pairwise interaction of charges (gravity, electrostatics).	Charge , Attractivity
Dipole	Pairwise dipole interaction (magnets).	Charge , Axis , Attractivity

Actuator – Thruster

```
body = phx.Body("Position", [0 0 1], "Shape", {"Capsule", "Axis", "z", "Density", 80});
phx.Thruster(body, "Point", [0 0 -0.9], "Direction", [0 0 1], ...
    "MaxThrust", 900, "Throttle", 1);
phx.Trace(body, "TracePoints", 240);    % motion trail
```

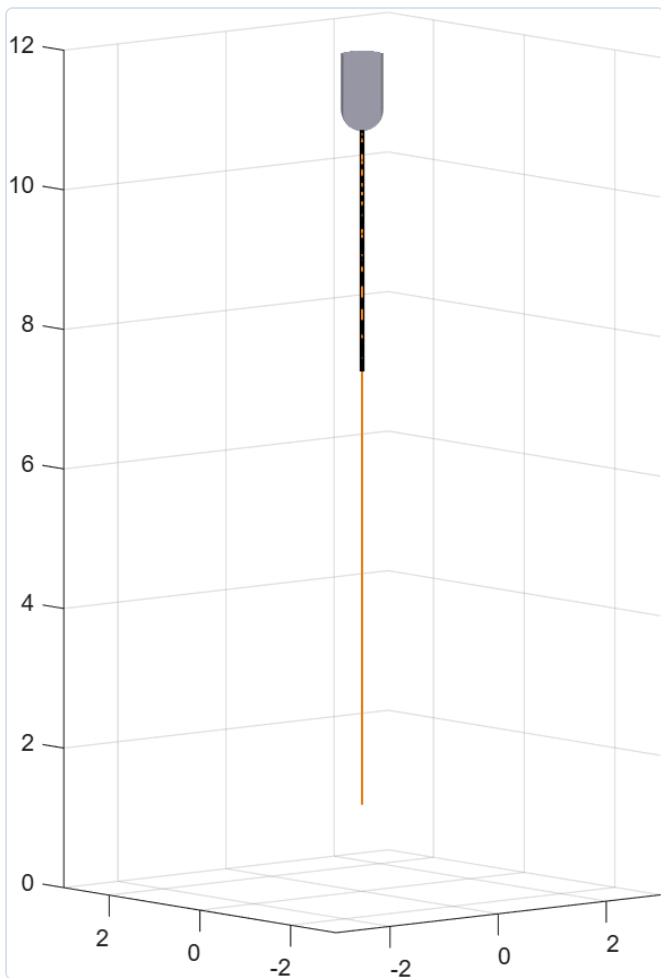


Fig. 5 – The thruster overcomes gravity and the body climbs; orange is the motion trail, the thrust vector is drawn below.

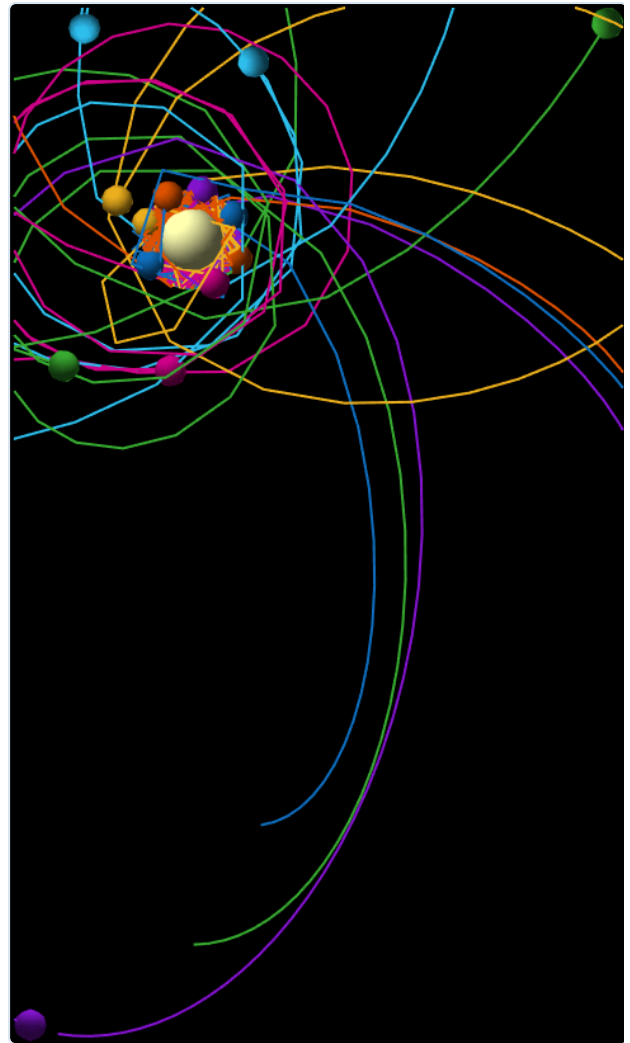


Fig. 6 – Bodies attracted by a central mass (**Monopole**) trace out orbits. The trails are drawn by **phx.Trace** .

Gravitational/electric field – Monopole

```
% mutual gravity of a group of bodies, the built-in gravity is turned off
phx.Monopole(planets, "Attractivity", 6.6743e-11, "Charge", [planets.Mass]);
sim = phx.Simulation(planets, "Gravity", 0);
```

10 Measuring and logging data

Simulation results are observed with three helpers:

- `phx.Logger` – records property values over time (position, velocity, force ...).
- `phx.Trace` – draws the trail of a chosen body point (see Figs. 3, 5, 6).
- `phx.Measure` – measures the distance and relative velocity between two bodies.

```
b = phx.Body("Position", [0 0 5], "Shape", {"Sphere"});  
phx.Body("Type", "static", "Shape", {"Box", "Size", [6 6 0.3]});  
  
% log the position at 100 Hz  
log1 = phx.Logger(b, "Parameters", "Position", "Frequency", 100);  
  
sim = phx.Simulation;  
sim.step(3, 600, 1);  
  
% plot the recording  
plot(log1.Time, log1.Data);  
xlabel("time [s]"); ylabel("position [m]"); legend("x","y","z");
```

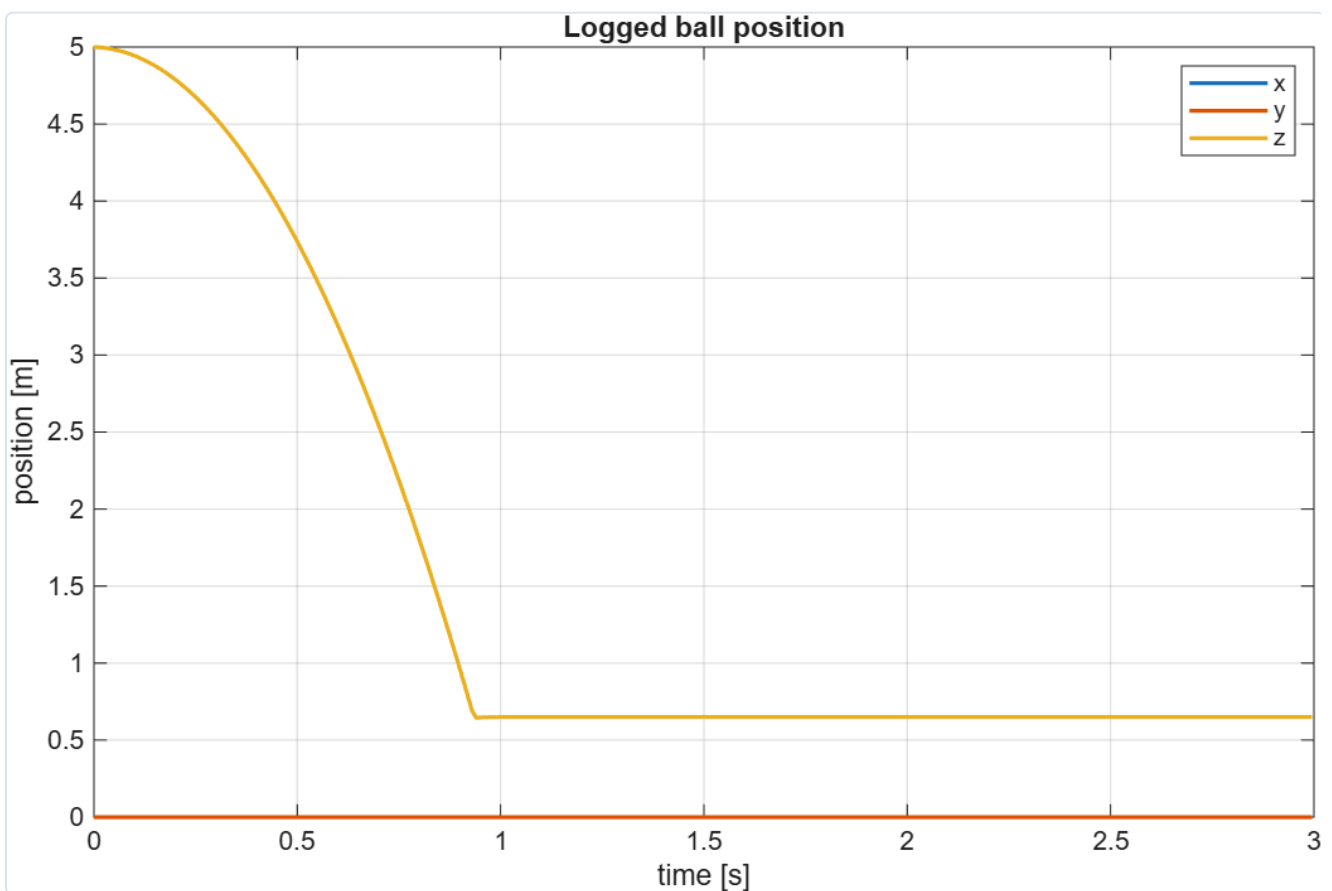


Fig. 7 – Logged vertical position of the falling ball: free fall and then rest on the floor. Data is read from `log1.Time` and `log1.Data`.

With multiple channels, individual signals are selected with `log.getChannel(k)`.

11 Automation and control: Script and Function

A scene need not just fall passively – you can insert your own logic that runs at every step. Two add-on objects exist for this.

phx.Script – a prescribed profile over time

Declarative (feedforward) control: to each property you assign either an **interpolated curve** (times + values), or a **function of time** given as a string. This is ideal for moving kinematic bodies – conveyors, arms, drive cams.

```
% steady rotation of a screw: EulerAngles as a function of time t
phx.Script(screw, {"EulerAngles", "[2*pi*t*0.5 0 0]"});

% prescribed path of an arm: interpolation of given points (pchip method)
phx.Script(arm, {"Position", time, position, "pchip"}, ...
             {"EulerAngles", time, angle});
```

phx.Function – a custom computation at every step

Inserts an arbitrary computation into the simulation loop without writing a custom class. The supplied callback is called every (sub)step with the signature `fcn(obj, parents, dt, time)` and acts by modifying the bound bodies directly. State between steps can be kept in `obj.UserData`. It is the ideal tool for **feedback control**, custom force laws or filters.

```
% a time-dependent periodic force
phx.Function(body, @(obj, p, dt, t) p{1}.applyForce([0 0 50]*sin(t)));

% a controller reading the scene state and driving several bodies (feedback)
phx.Function({ball, plate}, @(obj, p, dt, t) balanceLaw(p, Kp, Kd));
```

Which one when. Use `phx.Script` when you know the profile in advance. Use `phx.Function` when you need to react to the current state (read position/velocity and compute a control action). This exact feedback pattern is used by the closing self-balancing robot example (chapter 15).

12 What PHX can do – richer scenes

The same building blocks scale up to large scenes. Two examples from the shipped demos:

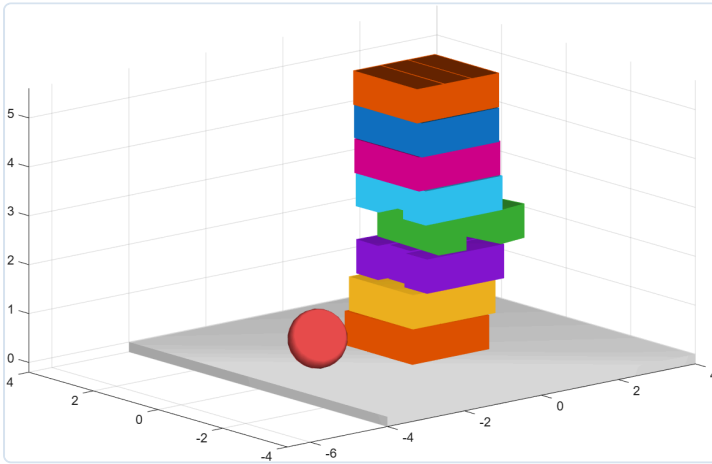


Fig. 8 – A stack of boxes (friction and contacts) hit by a flying ball.

See [phxex_jenga_free](#).

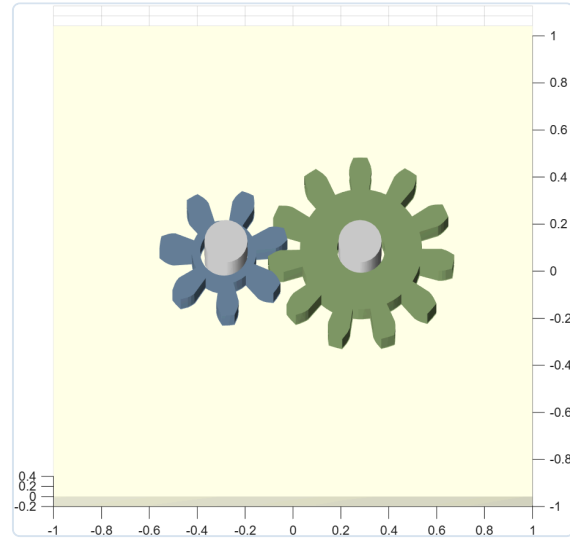


Fig. 9 – Two gears made from STL models mesh through their actual teeth. See [phxex_gears](#).

Other possibilities include several independent simulations side by side, multiple views of a single scene, textures and a sky, terrain, conveyors, suspended vehicles, feedback controllers, or optimization running without rendering (*headless*).

13 Interactive viewer

Instead of bare axes you can use the enhanced viewer `phx.extra.Viewer` with a sky, a movable camera and the ability to edit the scene with the mouse:

```
[~, ax] = phx.extra.Viewer("clear", "ViewMode", "sky");
% then add bodies into the ax axes as usual
phx.Body(ax, "Position", [0 0 2], "Shape", {"Sphere"});
```

Control	Action
Left mouse button	Pan the camera.
Middle button	Orbit the camera.
Wheel	Zoom; with Shift, the field of view.
Arrows / WASD	Pan and move the camera.
Double-click	Select / activate a body (edit mode).
F1 ... F5, Home	Help, light, view mode, free run, reset.

14 Ready-made examples (demos)

The examples are the best way to learn – you run them with a single command and study the source code. Recommended to begin with:

Command	What it shows
<code>phxex_minimal1</code>	The bare minimum – a body and a floor.
<code>phxex_shapes</code>	A gallery of all primitive shapes (Fig. 2).
<code>phxex_joints</code>	A hinge and a ball joint (Fig. 3).
<code>phxex_springs</code>	A spring system with bodies added and removed at runtime (Fig. 4).
<code>phxex_gravity</code>	Particles orbiting a star (Fig. 6).
<code>phxex_gears</code>	Two meshing gears (Fig. 9).
<code>phxex_jenga_free</code>	A tower collapsing – friction and contacts (Fig. 8).
<code>phxex_terrain</code> , <code>phxex_textures1</code>	Terrain and textures.
<code>phxex_buggy</code> , <code>phxex_conveyors</code>	A suspended vehicle, conveyors.

The same folder also holds more advanced scenes: `phxex_segway` (a self-balancing robot), `phxex_drone` (a quadcopter), `phxex_rocket` , `phxex_galton` (a Galton board), `phxex_magnets` (magnetic self-assembly) or `phxex_optimize` (optimization without rendering).

15 A complete example: self-balancing robot

To finish, an example that combines almost everything from the previous chapters into one working whole – a self-balancing two-wheeled robot (a segway). It shows what more complex work with PHX can look like.

The scene is built from ordinary building blocks:

- **Bodies** – a tall block (the body) and two wheels (cylinders), with the center of mass placed high above; the assembly is therefore an *unstable inverted pendulum* and would topple on its own.
- **Joints** – each wheel is connected to the body by a hinge (`RevoluteJoint`).
- **Contacts and friction** – the wheels propel the robot solely through frictional contact with the ground.
- **Feedback control** – at every step the body tilt and angular rate are read and a PD controller computes the wheel drive torque (exactly the pattern from chapter 11).
- **Logging** – `phx.Logger` records the body tilt for a later plot.

```
% --- core of the control loop (runs with a 5 ms step) ---
Kp = 4000; Kd = 380; dt = 0.005;
sim = phx.Simulation;
for k = 1:500
    pitch      = body.EulerAngles(2);      % sensor: body tilt
    pitchRate  = body.AngularVelocity(2);  % sensor: tilt rate
    tau = Kp*pitch + Kd*pitchRate;          % PD controller
    wheelL.applyTorque(-[0 0 tau], true);  % control action on the wheels
    wheelR.applyTorque(-[0 0 tau], true);
    sim.step(dt, 1, 1);                    % one simulation step
end
```

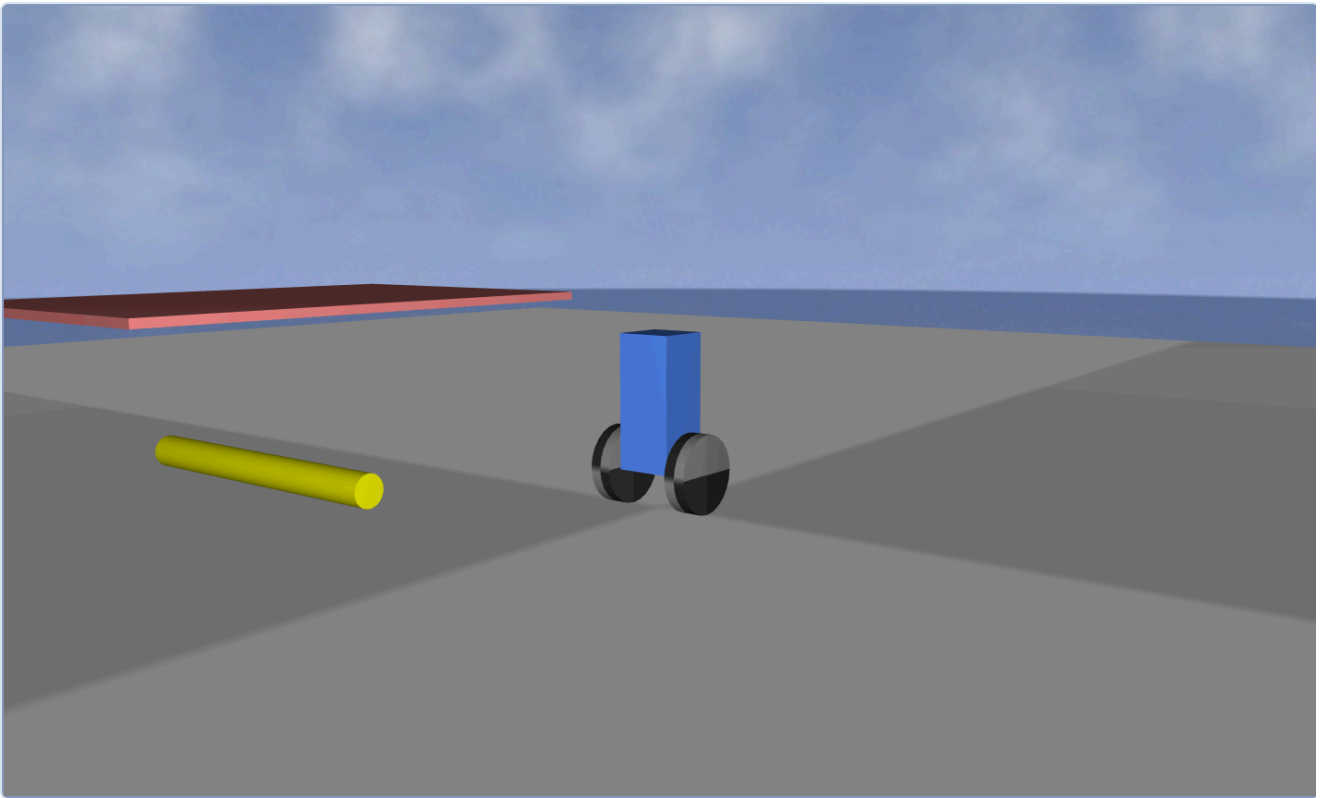


Fig. 10 – The shipped `phxex_segway` shown directly in the interactive PHX viewer (sky mode). The robot actively keeps itself upright; an obstacle (a ramp) included in the scene sits in the foreground.

The same approach – bodies × joints × sensors × a `phx.Function` /control loop – is behind advanced examples such as `phxex_drone` (a quadcopter), `phxex_balance` (balancing a ball on a plate) or `phxex_rocket`. Once you master this skeleton, you can build practically any controlled mechanism in PHX.

16 Quick reference

The skeleton of every scene

```
clf; view(3); axis equal; grid on; camlight headlight; % scene
phx.Body(...); % bodies
phx.Body(...);
sim = phx.Simulation; % engine
sim.step(interval, substeps, redrawStep); % run
delete(sim); % cleanup
```

The most common commands

Notation	Meaning
<code>phx.Body("Position",[x y z],"Shape",{...})</code>	A dynamic body.
<code>phx.Body("Type","static",...)</code>	An immovable body (floor, wall).
<code>phx.Simulation(bodies,"Gravity",g)</code>	Simulation of the given bodies with a gravity choice.
<code>sim.step(t, n, r)</code>	Simulate <code>t</code> s in <code>n</code> steps, redraw every <code>r</code> .
<code>sim.addObject(b)</code>	Add a body to a running simulation.
<code>phx.RevoluteJoint(A,B,"PointA",..,"PointB",..)</code>	A hinge.
<code>phx.Spring(A,B,"Stiffness",k,"Damping",d)</code>	A spring with a damper.
<code>phx.Logger(b,"Parameters","Position")</code>	Data logging (<code>.Time</code> , <code>.Data</code>).
<code>phx.Trace(b)</code>	A motion trail.
<code>phx.Function(b, @(o,p,dt,t) ...)</code>	Custom computation/control at every step.
<code>body.applyForce(F)</code> · <code>body.applyTorque(M)</code>	Acting with a force / torque.

Where to look next. Every class has help in MATLAB – try `help phx.Body` , `doc phx.Simulation` or `methods phx.Spring` . The fastest path to understanding, though, is to open and tweak one of the examples in the `examples/` folder.